

Fleet Transition Planner

The technology stack, explained for a C++ developer

Repository [The-Sims-Digipen/Fleet-Project](#) · branch [ui-vehicle-presets](#) · 15 September 2026

This document explains every technology in the repository and maps it onto the nearest C++ equivalent. The analogies are teaching aids, not claims of equivalence — each entry notes where the comparison breaks down.

1. The stack at a glance

Layer	Technology	Closest C++ analogue
Language	TypeScript 5.9 (on JavaScript)	A static type checker bolted onto an untyped language; types are erased before running
Runtime	Node.js 24 (server) / browser (client)	libc + libstdc++ + the OS loader
Packages	pnpm 11 workspace	vcpkg / Conan, plus a CMake super-build
Build	Vite 8, tsc, tsx	CMake + Ninja + linker; -fsyntax-only; an interpreter front-end
UI	React 19 + React DOM	Dear ImGui's declarative style over a Qt-style retained widget tree
Styling	Tailwind CSS 4	Composing style from a fixed vocabulary, with dead-style elimination at link time
3D	Three.js 0.185, React Three Fiber 9, drei 10	OGRE / bgfx over OpenGL ES; plus an alternate ImGui render backend
State	Zustand 5	An observable singleton with selector-based subscriptions
Server	Fastify 5	Crow / Drogon / cpp-httplib
Validation	Zod 4	Hand-written validators, but the static type is derived from the validator
Database	PostgreSQL, Drizzle ORM, postgres-js	PostgreSQL, sqlpp11 / ODB, libpq
Charts	ECharts 6 UNUSED	ImPlot / matplotlib-cpp
Server cache	TanStack Query 5 UNUSED	A caching, retrying wrapper around std::future
Tests	Vitest 4, Testing Library, jsdom	GoogleTest / Catch2, a UI automation harness, a stub platform layer

2. Language and runtime

JavaScript (ECMAScript 2023)

The only language browsers execute. Garbage collected, dynamically typed, single-threaded with an event loop. Every other language in a browser project is compiled down to it.

vs C++: no manual memory management, no pointers, no templates, no operator overloading, no threads by default. Objects are reference types with GC — there is no value semantics, no copy constructor, no destructor. The single biggest adjustment is that *nothing is deterministic at scope exit*: there is no RAII.

TypeScript

A static type layer over JavaScript. Config in `apps/client/tsconfig.app.json` runs in strict mode with `noUnusedLocals` and `noUnusedParameters`. Note `"noEmit": true` — `tsc` in this project only checks types and produces no output; Vite does the actual transpiling.

vs C++: closest to running `g++ -fsyntax-only` as a separate step from codegen. Two deeper differences matter day to day:

- **Types are erased.** They exist only at compile time and vanish in the shipped JavaScript. You cannot branch on a type at runtime the way you can with `RTTI` or `dynamic_cast`. The `erasableSyntaxOnly` flag enforces this.
- **Typing is structural, not nominal.** In C++, `struct A { int x; };` and `struct B { int x; };` are unrelated types. In TypeScript they are interchangeable. The nearest C++ feature is a C++20 *concept*, which also matches on shape rather than name.

Generics (`SelectControl<T extends string>` in `controls.tsx`) are templates, but type-checked once at the definition rather than per instantiation — so you get real error messages instead of template instantiation walls.

Node.js

The JavaScript runtime outside the browser: V8 plus file I/O, sockets, processes. It runs the Fastify server, the build tools, and the test runner.

vs C++: the runtime and standard library rolled together — `libc`, `libstdc++`, and the dynamic loader. Browsers deliberately withhold most of it: browser JavaScript has no filesystem and no raw sockets, which is why this project is split into `apps/client` and `apps/server` at all.

3. Build and dependency tooling

pnpm (workspace)

Package manager and monorepo driver. `pnpm-workspace.yaml` declares `apps/*` as sub-packages; the root `package.json` defines `pnpm dev`, `build`, `test` and `verify` which fan out to both. `pnpm-lock.yaml` (127 KB) pins the exact resolved version of every transitive dependency.

vs C++: `vcpkg` or `Conan` for acquisition, and a `CMake` super-build with `add_subdirectory` for the workspace. The lockfile is a `Conan` lockfile. `pnpm`'s distinguishing trick is a content-addressable global store that hard-links packages into each project instead of copying — conceptually a `ccache` or `vcpkg` binary cache, so ten projects sharing a dependency store it once.

Vite

Dev server and production bundler, configured in `apps/client/vite.config.ts` with the `React` and `Tailwind` plugins on port 5173.

vs C++: two modes that map to two familiar workflows.

- **Dev** (`pnpm dev:client`): serves each source file as a separate ES module, transformed on demand as the browser requests it, and hot-swaps changed modules without a page reload. This is incremental compilation plus `MSVC Edit and Continue` — you never re-link the world to see a change.
- **Build** (`pnpm build`): `Rollup` bundles and tree-shakes everything into optimised chunks. That is a full LTO link with `--gc-sections`. The large-chunk warning the README mentions is the 3D code producing a big output object.

tsx and concurrently

`tsx watch src/index.ts` runs the TypeScript server directly with no build step and restarts on change.
`concurrently` runs client and server dev processes side by side under one `pnpm dev`.

vs C++: `tsx` is `Cling`, or a shorthand for compile-and-run in one command. `concurrently` is just launching two background processes with unified, prefixed output.

4. The UI layer

React and JSX

The UI framework. A component is a plain function that takes props and returns a description of UI; `apps/client/src/components/` holds eight of them. JSX is the XML-looking syntax inside those functions — `<button onClick={...}>` compiles to an ordinary function call.

vs C++: React *looks* like Dear ImGui. You write one function that describes the whole widget from current state, and you call it again whenever state changes — no `setText()`, no `setEnabled()`, no manual widget mutation. But under the hood it behaves like Qt: React keeps a retained tree, diffs your new description against the old one, and applies only the differences. So you get ImGui's programming model with Qt's performance characteristics.

Hooks: `useState` is a member variable that survives between calls to the function, keyed by call order. `useEffect` and `useLayoutEffect` run side effects after render and may return a cleanup function — **this is the closest thing to a destructor in the language**, and `ModelObject.tsx` uses it exactly that way:
`useLayoutEffect(() => () => instance.dispose(), [instance]);`

React DOM

The renderer that turns React's abstract element tree into real browser DOM nodes. `main.tsx` mounts the app with `createRoot(...).render(<App />)`.

vs C++: a rendering backend behind a UI library — ImGui's D3D11 backend versus its Vulkan backend. React itself is backend-agnostic, which is precisely what makes React Three Fiber possible.

StrictMode: `main.tsx` wraps the app in `<StrictMode>`, which deliberately double-invokes components in development to surface impure code. It is a sanitiser build. The comment in `WorldObjects.tsx` about a queued dialog close event is a direct consequence.

Tailwind CSS

Styling by composing single-purpose utility classes directly in markup: `className="flex min-h-0 flex-col overflow-hidden bg-surface"`. The project's design tokens live in the `@theme` block of `apps/client/src/index.css` — `--color-accent: #55d6be` and friends.

vs C++: there is no clean analogue, but the shape of the idea is familiar. Instead of authoring a bespoke stylesheet per widget (Qt's QSS), you assemble appearance from a fixed vocabulary of named constants, and the build strips every utility you never used — `-ffunction-sections -Wl,--gc-sections` for CSS. The project convention in `AGENTS.md` is strict: `index.css` holds only global tokens and base rules; everything component-specific stays in the component.

5. The 3D layer

Three.js and WebGL

The 3D engine. `apps/client/src/models/van.ts` builds a 366-line low-poly van from `BoxGeometry`, `CylinderGeometry` and a hand-authored `BufferGeometry` for the sloped cab, using metres and Y-up.

vs C++: a scene-graph library over a GPU API — OGRE, or bgfx with extras. WebGL2 is essentially OpenGL ES 3.0, so the concepts transfer directly: `BufferGeometry` is a VAO plus its vertex and index buffers, `MeshStandardMaterial` is a PBR shader plus its uniforms, `Mesh` is a draw call, and `Group` is a transform node.

The one place GC does not save you. JavaScript's collector reclaims JavaScript objects, but GPU-side buffers, textures and compiled programs live in the driver and are invisible to it. They must be released by hand, exactly like `glDeleteBuffers`. That is what `models/proceduralModel.ts` exists for: it walks the group, records every geometry, material and texture, and frees them in `dispose()`. Forgetting it leaks VRAM in a way no profiler on the JS heap will show. This is the closest this codebase gets to RAI, and it is manual.

React Three Fiber

A React renderer that emits Three.js objects instead of DOM nodes. It is why `WorldScene.tsx` can write `<ambientLight intensity={...} />` and `<color attach="background" />` as if they were HTML tags.

vs C++: swapping ImGui's rendering backend while keeping the same UI code. Same declarative front end, entirely different output target — here, a `THREE.Scene` graph. The payoff is that scene contents become a pure function of the Zustand document: `WorldScene.tsx` maps over `document.objects` and R3F reconciles the scene graph to match.

drei

A helper collection for React Three Fiber. This project currently uses one piece of it: `<OrbitControls>` for camera orbit, pan and zoom.

vs C++: the extras or contrib module that ships alongside a graphics library — camera controllers, gizmos, loaders, debug helpers.

6. Application state

Zustand

The client state container. `apps/client/src/state/sceneStore.ts` holds the scene document, the editor selection, and a 100-deep undo/redo history, and exposes actions such as `addObject`, `updateTransform` and `undo`.

vs C++: a singleton with an observer list. The refinement is the *selector*: `useSceneStore((state) => state.document.light)` subscribes a component to one slice of state, so changing the light re-renders the lighting component and nothing else. That is a signal/slot connection scoped to a field rather than the whole object.

Two patterns you will recognise:

- **Memento** — undo works by snapshotting whole immutable documents into `history.past`. Cheap only because the document is deliberately tiny; the code says so in a comment.
- **Transaction** — `beginEdit`, `commitEdit` and `cancelEdit` collapse a whole slider drag into one undo step, with `cancelEdit` rolling back to the baseline on Escape. `controls.tsx` wires these to pointer capture and key events.

TanStack Query

INSTALLED, NOT YET USED

Intended for loading and saving projects against the API once F09 persistence exists.

vs C++: a cache in front of `std::future` — deduplicates identical in-flight requests, retries on failure, caches responses with a staleness policy, and exposes loading and error status. In C++ you would hand-roll this around a thread pool and a mutex-guarded map.

7. Server and data

Fastify

The HTTP server framework. `apps/server/src/app.ts` currently registers CORS and two routes, `GET /health` and `GET /api`; `index.ts` listens on port 3001.

vs C++: Crow, Drogon or cpp-httplib. Routes register handlers into a dispatch table; plugins such as `@fastify/cors` are middleware in a request chain. The structural difference is concurrency: Fastify handles thousands of connections on one thread via an event loop and async handlers, where a C++ server typically uses a thread pool or Asio.

Zod

Runtime schema validation, planned for validating project documents on both sides of the API.

vs C++: this is the entry with the most interesting gap. C++ has no reflection before C++26, so validating JSON means hand-writing a checker and separately declaring the struct, then keeping them in sync forever. Zod inverts it: you declare the *validator*, and the static type is derived from it with `z.infer<typeof Schema>`. One definition yields both a compile-time type and a runtime check that cannot drift apart. The nearest C++ equivalent is a Protobuf or FlatBuffers schema compiler — except the schema is ordinary code, not a separate file run through a generator.

Drizzle ORM and drizzle-kit

Typed SQL query builder and migration generator. `apps/server/src/db/schema.ts` is still an empty placeholder; `db:generate` and `db:migrate` scripts are wired but unused.

vs C++: sqlpp11 or ODB — you describe tables once and the query builder type-checks your SQL against them, so a renamed column fails at compile time rather than in production. `drizzle-kit generate` diffs your schema against the last migration and emits versioned SQL, which has no common C++ counterpart; it is closer to a database-focused code generator in your build step.

postgres-js, PostgreSQL, dotenv

`db/client.ts` wraps the postgres driver in Drizzle. `dotenv` loads `apps/server/.env` for `PORT` and `DATABASE_URL`.

vs C++: postgres is libpq or libpqxx — connection pooling and the wire protocol. PostgreSQL is the same PostgreSQL. `dotenv` reads a key=value file into the environment, like a small INI config loader that populates `getenv`.

ECharts and echarts-for-react

INSTALLED, NOT YET USED

Charting for the cost, emissions and roadmap views specified in F06.

vs C++: ImPlot, or matplotlib-cpp. You hand it a configuration object describing series, axes and tooltips; it draws and handles interaction.

8. Testing

Vitest

The test runner, 24 tests across five files, run by `pnpm test`. It shares Vite's config, so tests compile the same way the app does.

vs C++: GoogleTest or Catch2. `describe` and `it` are `TEST_F`, `expect(x).toBe(y)` is `EXPECT_EQ`, `beforeEach` is `SetUp()`. `vi.mock()` is a seam for substituting a dependency — link-time interposition, without the linker.

Testing Library and jsdom

`vitest.config.ts` sets `environment: "jsdom"`. Testing Library then queries the rendered UI the way a user would — `screen.getByRole("button", { name: "Add Object" })`.

vs C++: `jsdom` is a pure-JavaScript reimplementation of the browser DOM, so tests run headless with no browser — a stub platform layer, or a software rasterizer standing in for a GPU. Testing Library is a UI automation harness like Qt Test, but addressing widgets by accessible role and label rather than by pointer or object name, so refactoring markup does not break tests.

Its limit is worth knowing. `jsdom` implements no WebGL and no `<dialog>` top layer, so `App.test.tsx` stubs the entire 3D viewport with `vi.mock("./components/WorldScene")` and patches `showModal`. Rendering, picking and orbiting are therefore *not* covered by the test suite — the project docs require manual real-browser checks for those, and the README records when they were last done.

9. C++ idioms translated

C++	This codebase	Note
<code>#include "foo.h"</code>	<code>import { foo } from "./foo"</code>	Real modules, not textual inclusion. No header/implementation split, no include guards, no ODR.
Header + .cpp pair	One .ts file with export	The exports <i>are</i> the public interface.
namespace	The module itself	Nothing is global unless exported and imported.
Destructor / RAII	<code>useEffect</code> cleanup return	Runs on unmount, not scope exit. Non-deterministic everywhere else.
<code>unique_ptr</code> , <code>shared_ptr</code>	Garbage collection	Free for CPU memory. Does not apply to GPU resources — see <code>dispose()</code> .
<code>template<typename T></code>	Generics: <code><T extends string></code>	Checked once at definition; no instantiation per type, no SFINAE.
C++20 concepts	Structural types and interfaces	The closest match. TypeScript matches on shape, never on name.
<code>const</code>	<code>const</code> , <code>readonly</code>	JS <code>const</code> binds the reference, not the contents — like <code>T* const</code> , not <code>const T*</code> .
Value semantics, copy ctor	None; everything is a reference	Copies are explicit: <code>{...obj}</code> , <code>[...arr]</code> . types.ts ships <code>copyTransform()</code> for exactly this reason.
Threads, <code>std::mutex</code>	Single-threaded event loop	No data races by construction. Parallelism needs Web Workers and message passing.
C++20 coroutines, <code>std::future</code>	<code>async</code> / <code>await</code> , <code>Promise</code>	Nearly a direct match — <code>await</code> suspends and resumes on the same thread.
<code>try</code> / <code>catch</code>	<code>try</code> / <code>catch</code> , plus error boundaries	A React error boundary catches exceptions thrown while rendering a subtree. App.tsx and ModelObject.tsx each use one to isolate 3D failures.
CMake target	<code>package.json</code>	Declares name, dependencies and build/test commands for one unit.
<code>static_assert</code>	<code>satisfies</code> operator	catalog.ts uses <code>satisfies</code> <code>Record<string, ObjectDefinition></code> to check conformance while keeping the exact literal type.

10. What this means for F01 — Vehicle Presets

Your branch delivers a UI-only stub with mock data: list presets, create, edit, delete, and choose a 3D model. Of everything above, you need four things.

You need	Where	Why
React + TypeScript	a new component under <code>apps/client/src/components/</code>	The whole panel is one component holding mock data in <code>useState</code> .
Tailwind	<code>inline className</code>	Project rule: no component-specific CSS in <code>index.css</code> .
Existing shared controls	<code>components/controls.tsx</code> , <code>CollapsibleSection.tsx</code>	<code>NumberControl</code> and <code>SelectControl</code> already exist and match the editor's look and keyboard behaviour. Reuse beats reinvention here.
The catalog	<code>scene/catalog.ts</code>	<code>objectDefinitions</code> is the source of truth for the "3D model selection" field. Today it has exactly one entry, <code>van</code> .

You do **not** need `Three.js`, `Zustand`, `Fastify`, `Drizzle`, `Zod` or `ECharts` for this feature. Per **AGENTS.md**, the panel must be a self-contained feature component that `Sidebar.tsx` merely composes — no preset logic inside `Sidebar` itself.

Branch status as of this document: `ui-vehicle-presets` exists locally and on `origin`, but contains no commits beyond `main` and an empty diff against it. Work has not started.

11. Summary

The stack is conventional modern web, and most of it maps onto something you already know. Four points are worth carrying forward:

- **Types are compile-time only.** TypeScript vanishes at runtime; `Zod` exists because of that gap, and validating anything crossing the network boundary is not optional.
- **GC covers memory, not the GPU.** `Three.js` resources need explicit `dispose()`. The project has already centralised this in `proceduralModel1.ts` — route new models through it.
- **React is ImGui's model with Qt's machinery.** Describe the UI from state and let the diff handle updates; never mutate widgets by hand.
- **About a third of the declared stack is aspirational.** `ECharts`, `TanStack Query`, `Drizzle`, `Zod` and `PostgreSQL` are installed and documented but not yet used in any source file. The architecture docs describe M1–M6; the code today is a 3D scene-editor prototype.